Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Модели данных и системы управления базами данных»

ОТЧЕТ
к лабораторной работе №3
на тему:
«РАБОТА СО СХЕМАМИ ДАННЫХ»
БГУИР 6-05-0612-02 67

Выполнил студент группы 353503
КОХАН Артём Игоревич

_____
(дата, подпись студента)

Проверил ассистент каф. информатики
ИОВИЧ Евгения Александровна

_____
(дата, подпись преподавателя)

Минск 2026

1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

1 Написать процедуру/функцию, на вход которой подаются два текстовых параметра (dev_schema_name, prod_schema_name), которые являются названиями схем баз данных (условно схема для разработки (Dev) и промышленная схема (Prod)). На выход процедура должна предоставить перечень таблиц, которые есть в схеме Dev, но нет в Prod, либо в которых различается структура таблиц. Наименования таблиц должны быть отсортированы в соответствии с очередностью их возможного создания в схеме prod (необходимо учитывать foreign key в схеме). В случае закольцованных связей выводить соответствующее сообщение.

2 Доработать предыдущий скрипт с учетом возможности сравнения не только таблиц, но и процедур, функций, индексов, пакетов.

3 Доработать предыдущий скрипт с генерацией ddl-скрипта на обновление объектов, а также с учетом необходимости удаления в схеме prod объектов, отсутствующих в схеме dev.

2 КРАТКИЕ ТЕОРИТИЧЕСКИЕ СВЕДЕНИЯ

Схема базы данных представляет собой совокупность объектов (таблиц, индексов, процедур, функций, пакетов, последовательностей, триггеров и других), принадлежащих конкретному пользователю. В процессе разработки обычно существует две основные среды: среда разработки (Dev), где вносятся изменения и добавляются новые функции, и промышленная среда (Prod), где работает стабильная версия приложения.

Oracle предоставляет несколько инструментов для работы со схемами, включая системные представления словаря данных, которые содержат метаинформацию обо всех объектах базы данных. Пакет DBMS_METADATA является инструментом для получения DDL-определений объектов, позволяя извлекать SQL-команды, необходимые для воссоздания объектов.

Процесс сравнения схем включает несколько этапов: идентификация объектов в каждой схеме, анализ их структуры и свойств, определение различий и генерация скриптов для приведения схем к одинаковому состоянию.

DDL-скрипты (Data Definition Language) используются для определения структуры базы данных и включают команды CREATE, ALTER, DROP для различных объектов. При генерации миграционных скриптов необходимо учитывать не только добавление новых объектов, но и удаление устаревших, а также модификацию существующих.

Методология сравнения должна охватывать все типы объектов: таблицы (сравнение столбцов, типов данных, ограничений), индексы (типы, столбцы, параметры), программные объекты (процедуры, функции, пакеты – сравнение исходного кода), последовательности (текущие значения, инкременты) и другие объекты.

3 ВЫПОЛНЕНИЕ РАБОТЫ

В ходе выполнения лабораторной работы была разработана система для сравнения и миграции схем баз данных Oracle между средами разработки (Dev) и промышленной (Prod). После запуска Oracle были созданы две тестовые схемы, имитирующие среды разработки и промышленной эксплуатации.

Листинг 3.1 – Создание двух тестовых схем

```
CREATE USER c##dev_schema IDENTIFIED BY dev123;
CREATE USER c##prod_schema IDENTIFIED BY prod123;
GRANT CONNECT, RESOURCE TO c##dev_schema, c##prod_schema;
ALTER USER c##dev_schema QUOTA UNLIMITED ON USERS;
ALTER USER c##prod_schema QUOTA UNLIMITED ON USERS;
```

В процессе разработки активно использовались сессии SQL*Plus для подключения к базе данных. Основная сессия разработчика осуществлялась через команду sqlplus sys/123@XE as sysdba, что позволяло работать с базой данных с правами администратора (SYSDBA). Для тестирования созданных процедур использовались отдельные сессии. На рисунке 3.1 видно, имеющиеся сессии и их схемы.

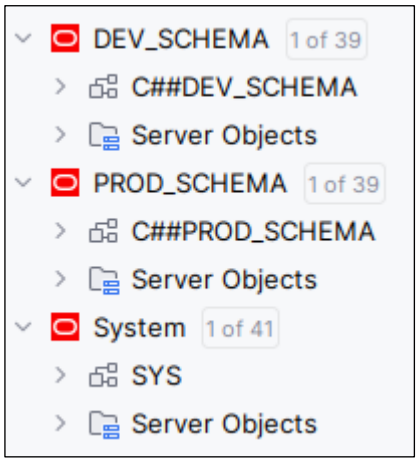

Рисунок 3.1 – Сессий и схемы в базе данных

Первым компонентом системы была создана функция get_object_ddl, которая использует пакет DBMS_METADATA для получения DDL-определений объектов. Основная процедура compare_schemas_complete выполняет комплексное сравнение двух схем. На рисунке 3.2 процесса выполнения видно, как процедура последовательно анализирует различные типы объектов: таблицы, индексы, процедуры, функции, пакеты и последовательности. Результаты сравнения выводятся в консоль в виде структурированного отчета, где явно видны различия между схемами C##DEV_SCHEMA и C##PROD_SCHEMA.

```
Dev схема: C##DEV_SCHEMA
Prod схема: C##PROD_SCHEMA
==================================================
1. ТАБЛИЦЫ:
Dev: 3 таблиц
Prod: 2 таблиц
+ PROJECTS (только в Dev)
2. ИНДЕКСЫ:
+ IDX_EMP_DEPT (только в Dev)
+ IDX_EMP_NAME (только в Dev)
+ IDX_PROJ_DATES (только в Dev)
3. ПРОЦЕДУРЫ/ФУНКЦИИ:
+ GET_DEPT_SALARY_TOTAL (FUNCTION, только в Dev)
+ HR_UTILS (PACKAGE, только в Dev)
+ HR_UTILS (PACKAGE BODY, только в Dev)
4. ПОСЛЕДОВАТЕЛЬНОСТИ:
+ SEQ_DEPT (только в Dev)
+ SEQ_EMP (только в Dev)
+ SEQ_PROJ (только в Dev)
5. ОБЩИЕ РАЗЛИЧИЯ:
Всего различий: 10
```

Рисунок 3.2 – Различия между схемами

Процедура generate_migration_script генерирует полный миграционный скрипт на основе выявленных различий. В ходе работы процедуры создается подробный скрипт, включающий несколько секций:

1 Удаление объектов, отсутствующих в Dev-схеме.

2 Создание новых таблиц.

3 Создание процедур и функций.

4 Создание пакетов.

5 Создание последовательностей.

6 Создание индексов.

На рисунке 3.3 видно, как генерируются DDL-команды для различных объектов. Например, для таблицы PROJECTS создается полное определение с указанием столбцов, типов данных, первичного ключа и внешнего ключа на таблицу EMPLOYEES.

```
-- СКРИПТ МИГРАЦИИ
-- Dev: C##DEV_SCHEMA
-- Prod: C##PROD_SCHEMA
-- ==========================================
-- 1. УДАЛЕНИЕ ОБЪЕКТОВ, ОТСУТСТВУЮЩИХ В DEV
DROP SEQUENCE SEQ_DEPT_PROD;
-- 2. СОЗДАНИЕ НОВЫХ ТАБЛИЦ
CREATE TABLE "C##DEV_SCHEMA"."PROJECTS"
(   "PROJECT_ID" NUMBER,
    "PROJECT_NAME" VARCHAR2(100),
    "MANAGER_ID" NUMBER,
    "START_DATE" DATE,
    "END_DATE" DATE,
     PRIMARY KEY ("PROJECT_ID")
USING INDEX PCTFREE 10 INITRANS 2 MAXTRANS 255 COMPUTE STATISTICS
STORAGE(INITIAL 65536 NEXT 1048576 MINEXTENTS 1 MAXEXTENTS 2147483645
PCTINCREASE 0 FREELISTS 1 FREELIST GROUPS 1
BUFFER_POOL DEFAULT FLASH_CACHE DEFAULT CELL_FLASH_CACHE DEFAULT)
TABLESPACE "USERS"  ENABLE,
     FOREIGN KEY ("MANAGER_ID")
      REFERENCES "C##DEV_SCHEMA"."EMPLOYEES" ("EMP_ID") ENABLE
) SEGMENT CREATION IMMEDIATE
PCTFREE 10 PCTUSED 40 INITRANS 1 MAXTRANS 255
NOCOMPRESS LOGGING
STORAGE(INITIAL 65536 NEXT 1048576 MINEXTENTS 1 MAXEXTENTS 2147483645
PCTINCREASE 0 FREELISTS 1 FREELIST GROUPS 1
BUFFER_POOL DEFAULT FLASH_CACHE DEFAULT CELL_FLASH_CACHE DEFAULT)
TABLESPACE "USERS"
```

Рисунок 3.3 – DDL-команды для таблицы PROJECTS

Для функций и процедур генерируется полный исходный код с параметрами и логикой выполнения, смотреть рисунок 3.4.

```
-- 3. ПРОЦЕДУРЫ И ФУНКЦИИ
CREATE OR REPLACE EDITIONABLE FUNCTION "C##DEV_SCHEMA"."GET_DEPT_SALARY_TOTAL" (p_dept_id NUMBER)
RETURN NUMBER AS
v_total NUMBER;
BEGIN
SELECT SUM(salary) INTO v_total
FROM employees
WHERE dept_id = p_dept_id;
RETURN NVL(v_total, 0);
END;
/
CREATE OR REPLACE EDITIONABLE PROCEDURE "C##DEV_SCHEMA"."HIRE_EMPLOYEE" (
p_first_name VARCHAR2,
p_last_name VARCHAR2,
p_dept_id NUMBER,
p_salary NUMBER DEFAULT 50000
) AS
BEGIN
INSERT INTO employees (emp_id, first_name, last_name, dept_id, salary, hire_date)
VALUES (seq_emp.NEXTVAL, p_first_name, p_last_name, p_dept_id, p_salary, SYSDATE);
COMMIT;
DBMS_OUTPUT.PUT_LINE('Employee hired: ' || p_first_name || ' ' || p_last_name);
END;
```

Рисунок 3.4 – DDL-команды для функций и процедур

Ещё одним важным аспектом работы с сессиями было тестирование процедуры, которая определяет оптимальный порядок создания таблиц с учетом внешних ключей. Процедура анализирует зависимости и выводит рекомендации по порядку создания. На рисунке 3.5 видно, как процедура определяет, что таблица PROJECTS должна создаваться после таблицы EMPLOYEES из-за наличия внешнего ключа.

```
=== ПОРЯДОК СОЗДАНИЯ ТАБЛИЦ ДЛЯ СХЕМЫ C##DEV_SCHEMA ===
1. PROJECTS (без внешних ключей)
2. DEPARTMENTS (с внешними ключами)
3. EMPLOYEES (с внешними ключами)
```

Рисунок 3.5 – Оптимальный порядок создания таблиц

ВЫВОД

В ходе выполнения лабораторной работы были успешно изучены и применены на практике методы сравнения схем баз данных и генерации миграционных скриптов в Oracle. Разработана система для автоматического анализа различий между средами разработки (Dev) и промышленной (Prod).

Была создана функция get_object_ddl, которая использует пакет DBMS_METADATA для получения DDL-определений различных объектов базы данных, включая таблицы, индексы, последовательности, процедуры, функции и пакеты. Эта функция обеспечивает основу для автоматизированного извлечения метаданных объектов схем.

Разработана процедура compare_schemas_complete, выполняющая комплексное сравнение двух схем Oracle. Процедура анализирует все основные типы объектов, выявляет различия в их наличии и структуре, предоставляя детализированный отчет о выявленных расхождениях. На примере сравнения схем C##DEV_SCHEMA и C##PROD_SCHEMA было установлено наличие 10 различий в объектах, включая дополнительные таблицы, индексы, программные объекты и последовательности.

Создана процедура generate_migration_script, которая автоматически генерирует полный миграционный скрипт для приведения Prod-схемы в соответствие с Dev-схемой. Скрипт включает как удаление устаревших объектов, так и создание недостающих.

Реализована процедура get_table_creation_order, определяющая оптимальный порядок создания таблиц с учетом зависимостей по внешним ключам. Этот механизм предотвращает ошибки нарушения целостности данных при выполнении миграционных операций и автоматически выявляет потенциальные циклические зависимости.

В процессе работы была успешно созданы две схемы для имитации сред разработки и производства, отработаны различные сценарии подключения через SQL*Plus сессии с различными уровнями привилегий.

Полученные знания и навыки позволяют эффективно использовать механизмы Oracle для автоматизации процессов миграции баз данных, сравнения схем и управления изменениями.